



# Fleet Operations: A Lambda-Architecture Data Pipeline for Ride-Hailing Utilisation and Profitability

*answering a live question and a financial question from one immutable  
master dataset*

A mini-project report submitted to the

Department of Electrical and Information Engineering  
Faculty of Engineering  
University of Ruhuna  
Sri Lanka

in partial fulfilment of the requirements of

**EC8203 — Applied Big Data Engineering**

by

Ahamed N.F.F. - EG/2021/4390  
Safeer S.M. - EG/2021/4771  
Nafla M.N.P. - EG/2021/4687

Use Case 1: Ride-Hailing Fleet Operations  
Academic Year 2026 | Mini-Project Report

---

# Abstract

A ride-hailing operator needs two answers that do not fit the same processing model. A dispatcher needs to know which vehicles are idle and which zones are earning *now*, within seconds, and does not mind if one trip is missing. Finance needs to know which vehicles are becoming unprofitable once fuel and maintenance costs are applied, and that figure must be exact, complete, and restatable when a fuel partner resubmits a corrected file days later. This report presents a data platform that answers both from one immutable master dataset.

We chose a **Lambda architecture** and rejected Kappa on the record. What decided it was not latency but *recomputation cost*. The correction that forces a restatement never enters the event stream at all. It arrives on a separate channel, as a daily CSV drop, long after the telemetry it restates, so replaying the log, which is Kappa's entire recomputation story, does not touch the thing that actually causes restatements. A date-partitioned Parquet lake lets the batch layer read the single partition it needs and reconcile a full simulated day in 9.4 seconds. To offer the same guarantee, Kappa would need Kafka retention measured in months. Lambda's best-known weakness is two codebases that drift, and we answered it with a shared `common/` package that the simulators, both processing layers and the serving API all import. Two duplications remain, each kept in step with its Python counterpart by a test.

The system ingests a Kafka telemetry stream from a 50-vehicle simulated fleet plus a daily partner expense file, on a clock where one real second is one simulated minute. Spark Structured Streaming archives the raw stream and computes live views. Airflow runs a nine-task PySpark reconciliation per simulated day. PostgreSQL serves both layers behind a FastAPI surface that labels every figure `source: speed` or `source: batch` and never blends the two.

The central evidence is quantitative, and it does not flatter us. The speed layer runs a 10-simulated-minute watermark against injected lateness of up to 20 minutes, so the two layers are *built* to disagree. Across four reconciled days the revenue drift came out at 0.000000, 0.003384, 0.000000 and 0.000000. The one non-zero day traces to a single late `trip_end` event that the watermark dropped and the batch layer recovered. We then worked out *why* the drift is usually zero: 15 rows dropped per day, of which 3.4% are revenue-bearing, giving an expectation of about half an event. We report that arithmetic rather than inflating the fault rates to manufacture a bigger number. Recomputation was tested directly. A corrected v2 expense file reproduced the same 50 rows with different values, and a forced replay returned bit-identical figures.

Observability spans all five pipeline stages. Of ten alert rules, five have been seen firing *and* arriving end to end, and a sixth is suppressed on purpose by an inhibition rule. 202 automated tests and a 28-check smoke test pass against the live stack from a fresh clone. We also report six defects found by running the system rather than

---

reading it, among them a green healthcheck sitting over an archiver that had stopped writing 26 minutes earlier.

---

# Acknowledgements

We thank the staff of the Department of Electrical and Information Engineering, University of Ruhuna, for the module and the environment in which a project of this scope could be attempted inside two weeks, and the lecturer in charge of EC8203 for a brief that was specific enough to argue with — the tension between its two sub-questions is what made the architecture decision worth making rather than merely worth stating.

This system is assembled almost entirely from open-source work. We thank the maintainers of Apache Kafka, Apache Spark, Apache Airflow, PostgreSQL, FastAPI, Prometheus, Alertmanager, Grafana and Docker. We also thank the authors whose published arguments about Lambda and Kappa we engage with in [Section 2.1](#); disagreeing with a well-made case is a far better way to learn an architecture than accepting one.

# Contents

|                                                               |            |
|---------------------------------------------------------------|------------|
| <b>Abstract</b>                                               | <b>i</b>   |
| <b>Acknowledgements</b>                                       | <b>iii</b> |
| <b>Contents</b>                                               | <b>v</b>   |
| <b>List of Figures</b>                                        | <b>vi</b>  |
| <b>List of Tables</b>                                         | <b>vi</b>  |
| <b>Acronyms</b>                                               | <b>vii</b> |
| <b>1 Introduction</b>                                         | <b>1</b>   |
| 1.1 Background and Problem Statement . . . . .                | 1          |
| 1.2 Objectives and Scope . . . . .                            | 2          |
| <b>2 Architecture and Technology Stack</b>                    | <b>2</b>   |
| 2.1 Background: Lambda, Kappa, and the Critique . . . . .     | 2          |
| 2.1.1 Event Time, Watermarks and Delivery Semantics . . . . . | 3          |
| 2.2 The Decision . . . . .                                    | 4          |
| 2.3 Why Kappa Was Rejected . . . . .                          | 4          |
| 2.4 What Lambda Costs, and Our Mitigation . . . . .           | 5          |
| 2.5 Technology Stack . . . . .                                | 5          |
| 2.6 The Simulated Clock and the Data Contract . . . . .       | 5          |
| <b>3 Implementation</b>                                       | <b>7</b>   |
| 3.1 Ingestion . . . . .                                       | 7          |
| 3.1.1 Partitioning Is a Correctness Decision . . . . .        | 7          |
| 3.1.2 Fault Injection . . . . .                               | 7          |
| 3.2 The Speed Layer . . . . .                                 | 8          |
| 3.2.1 The Archiver Does as Little as Possible . . . . .       | 8          |
| 3.2.2 Stateful Idle Alerts . . . . .                          | 8          |
| 3.2.3 Idempotent Sinks . . . . .                              | 9          |
| 3.3 The Batch Layer and Orchestration . . . . .               | 9          |
| 3.3.1 The Batch Layer Trusts Nothing . . . . .                | 9          |
| 3.3.2 Schedule and Recomputation . . . . .                    | 9          |
| 3.4 Storage and Serving . . . . .                             | 10         |
| 3.4.1 The Lambda Merge Is Explicit, Never Blended . . . . .   | 10         |
| 3.5 Observability . . . . .                                   | 10         |
| 3.5.1 Alerting Decisions Worth Defending . . . . .            | 11         |
| <b>4 Results and Discussion</b>                               | <b>11</b>  |
| 4.1 The Consistency Argument . . . . .                        | 11         |
| 4.2 Recomputation Demonstrated . . . . .                      | 13         |
| 4.3 Verification . . . . .                                    | 13         |
| 4.4 Discussion . . . . .                                      | 13         |
| 4.4.1 What Running It Taught Us . . . . .                     | 13         |
| 4.4.2 Limitations . . . . .                                   | 14         |
| 4.4.3 At Production Scale . . . . .                           | 14         |
| <b>5 Conclusions and Recommendations</b>                      | <b>15</b>  |

## CONTENTS

---

|                                         |    |
|-----------------------------------------|----|
| References                              | 16 |
| A Repository Layout                     | 17 |
| B Database Schema                       | 18 |
| C Alert Rule Inventory                  | 20 |
| D API Surface                           | 20 |
| E Statement of Individual Contributions | 21 |
| F Defects Found by Operating the System | 23 |
| G At Production Scale                   | 23 |

# List of Figures

|     |                                       |    |
|-----|---------------------------------------|----|
| 2.1 | System architecture . . . . .         | 4  |
| 4.1 | Why the two layers disagree . . . . . | 12 |

# List of Tables

|     |                                                                                                                                                                               |    |
|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1.1 | The brief contains two sub-questions with incompatible requirements. No single processing model serves both well. . . . .                                                     | 2  |
| 2.1 | Technology selection, with the rejected alternative for each layer. . . . .                                                                                                   | 6  |
| 3.1 | Injected fault rates, configured against measured, and what each fault demonstrates. . . . .                                                                                  | 8  |
| 4.1 | Reconciliation outcomes over four consecutive simulated days. Revenue drift is the relative difference between the speed layer's and the batch layer's daily revenue. . . . . | 12 |
| A.1 | Top-level layout of the submitted repository. . . . .                                                                                                                         | 18 |
| B.1 | Serving-store tables, by owning layer. . . . .                                                                                                                                | 19 |
| C.1 | The ten Prometheus alert rules. <i>Verified</i> means observed firing <b>and</b> delivered into <code>alert_notifications</code> . . . . .                                    | 19 |
| D.1 | The eleven documented endpoints, by the layer that answers them. . . . .                                                                                                      | 21 |
| E.1 | Principal areas of responsibility by team member. . . . .                                                                                                                     | 22 |
| F.1 | Defects found by operating the system, and what each generalises to. . . . .                                                                                                  | 24 |
| G.1 | Simplifications made here, and what each becomes at production scale. . . . .                                                                                                 | 25 |

# Acronyms

|       |   |                                   |      |   |                               |
|-------|---|-----------------------------------|------|---|-------------------------------|
| API   | - | Application Programming Interface | RF   | - | Replication Factor            |
| CSV   | - | Comma-Separated Values            | RPC  | - | Remote Procedure Call         |
| CU    | - | Currency Unit (simulated)         | SLA  | - | Service-Level Agreement       |
| DAG   | - | Directed Acyclic Graph            | SLO  | - | Service-Level Objective       |
| DLQ   | - | Dead-Letter Queue                 | SQL  | - | Structured Query Language     |
| DQ    | - | Data Quality                      | SVG  | - | Scalable Vector Graphics      |
| ETL   | - | Extract, Transform, Load          | UDF  | - | User-Defined Function         |
| GPS   | - | Global Positioning System         | UUID | - | Universally Unique Identifier |
| HTTP  | - | Hypertext Transfer Protocol       | YAML | - | YAML Ain't Markup Language    |
| JSON  | - | JavaScript Object Notation        | ZK   | - | ZooKeeper                     |
| JVM   | - | Java Virtual Machine              |      |   |                               |
| KRaft | - | Kafka Raft (metadata mode)        |      |   |                               |

# Chapter 1

## Introduction

A ride-hailing operator runs two businesses at once. One is an operations business, conducted minute by minute: where are the cars, how many are earning, which driver has been sitting still for an hour. The other is a financial business, conducted a day at a time: what did each vehicle earn, what did it cost to run, and is it still worth keeping on the road. This project builds one platform that answers both, and argues that the interesting decision is not how to answer either but how to answer both without letting the fast answer contaminate the exact one.

### 1.1 Background and Problem Statement

The brief [1] specifies two ingestion paths that behave very differently. A streaming source emits GPS and telemetry events per vehicle every few seconds (`trip_id`, `driver_id`, `vehicle_id`, latitude, longitude, speed, status, fare, timestamp). A daily batch source drops one file per simulated day carrying expense records from garages and fuel partners (`vehicle_id`, `fuel_cost`, `maintenance_cost`, `distance_covered`, `service_flag`).

They differ in more than cadence. The telemetry stream is high-volume, continuous, owned by us, and tolerant of small losses. The expense file is low-volume, periodic, owned by a third party, and *authoritative about money*. It can also be wrong and be corrected: a fuel-card partner that mis-keys a reconciliation resubmits days later, and the figures published in the meantime must be restated. That asymmetry, more than any latency requirement, drives everything that follows.

The brief poses what looks like one business question: *what is fleet utilisation and earnings by area and time of day right now, and which vehicles are becoming unprofitable once yesterday's fuel and maintenance costs are factored in?* It is really two, and their requirements conflict, as Table 1.1 sets out.

A dispatcher deciding where to send a car needs an answer in seconds and does not care if one trip is missing. A finance analyst deciding whether to retire a vehicle needs a figure that is complete, exact and *reproducible*. Serving both from one processing model means either making the dispatcher wait for exactness they do not need, or letting finance quote a number whose error is unbounded and unknown. The problem is to provide both guarantees from one dataset while making it impossible for a consumer to confuse them.

Table 1.1: The brief contains two sub-questions with incompatible requirements. No single processing model serves both well.

|                   | Live utilisation          | Daily profitability                                    |
|-------------------|---------------------------|--------------------------------------------------------|
| Latency required  | seconds                   | next morning is acceptable                             |
| Accuracy required | approximate is fine       | exact; it is money                                     |
| Completeness      | best effort               | must cover the whole day                               |
| Recomputable?     | no need                   | <b>yes</b> , because partners resubmit corrected files |
| Consumer          | dispatcher watching a map | finance, making retire/keep decisions                  |

## 1.2 Objectives and Scope

We set out to choose between Lambda and Kappa on the evidence of this use case, and to say exactly why the loser lost. Both ingestion paths are simulated Python sources that inject faults into their own output, so the error handling can be shown working instead of claimed. The processing does real work on the data: cleaning, zone enrichment, windowed aggregation, stateful alerting, and a join across the two sources. Results land in a queryable store behind a live dashboard and a consolidated daily report. Every stage is instrumented, and what is instrumented includes the pipeline’s *known imprecision*, not just its health. That last point turned out to be the most interesting part of the project, so we measured the imprecision rather than assuming a value for it.

Three simplifications follow from running the whole system on a single 16 GB laptop from one command. Simulated time is compressed 60-fold, so one real second is one simulated minute and a simulated day takes 24 real minutes (Section 2.6). Kafka runs as a single broker with replication factor 1. Batch Spark runs in local mode, so the project demonstrates distributed *streaming* but not distributed *batch*. Authentication and multi-tenancy are out of scope. Section 4.4.3 states what each would become in production.

# Chapter 2

## Architecture and Technology Stack

### 2.1 Background: Lambda, Kappa, and the Critique

Marz and Warren propose Lambda as an answer to a specific problem: no single system is simultaneously fast, exact and fault-tolerant, so use three parts [2]. An

*immutable master dataset* records raw facts and is never updated in place; a *batch layer* recomputes views over it from scratch; a *speed layer* computes incremental, approximate views over the recent data the batch layer has not yet covered. Two properties of that construction matter here. The batch views are a pure function of an immutable input, so a bug in view computation is repaired by fixing the code and recomputing, never by editing derived state in place. Helland makes the general version of the argument: immutability turns a class of distributed-state problems into append-only bookkeeping [3]. And the speed layer's error is *temporary* by construction, discarded once the batch view catches up. Kleppmann places both in a wider treatment of derived data, where keeping a system of record distinct from a derived view is what makes recomputation safe at all [4].

Kreps's critique is the standard case against Lambda, and it deserves to be taken at full strength [5]. He does not argue that Lambda is incorrect. He argues that it is expensive in the wrong currency: two implementations of one business rule doubles the code you must reason about and, worse, gives you two implementations that can quietly disagree. His replacement, Kappa, drops the batch layer. One stream processor remains, and recomputation means *replaying the log* into a new version of the job.

As far as it goes, the critique holds. It rests on an assumption that is easy to miss, though, and this use case breaks it: that everything needed to recompute a result is *in the log*. Where that is true, replay is a complete recomputation story and the batch layer really is redundant machinery. The decision rule falls out of it. Choose Lambda when recomputation cannot be expressed as a log replay. In practice the rule is stated more often than applied, because most published comparisons end up weighing operational simplicity against latency, and that is a tie the fashionable option tends to win.

### 2.1.1 Event Time, Watermarks and Delivery Semantics

Three results from the stream-processing literature underpin the correctness argument in Chapter 4.

Akidau et al. formalise **event time versus processing time**. An event's event time is when it happened; its processing time is when the system saw it; and the two diverge arbitrarily under network delay, retries and partition skew [6]. Group an aggregation by processing time and its results depend on infrastructure behaviour rather than on the world, which for a financial figure rules it out.

**A watermark is a bet, not a guarantee.** Because a system cannot know whether more data will arrive for a past window, it must decide when to stop waiting. Dataflow and MillWheel make the same point: the watermark trades completeness against latency, and events later than it are *dropped*, not deferred [6, 7]. This is the mechanism measured in Section 4.1. We set our watermark tighter than the lateness the simulator injects, which makes the loss real and countable instead of hypothetical.

**End-to-end exactly-once requires idempotent sinks.** Structured Streaming guarantees exactly-once with respect to its own state and offsets through checkpointing [8], building on the micro-batch recovery model of discretised streams [9]. That guarantee stops at the sink: an external sink written from `foreachBatch` observes *at-least-once*, because a failure between writing and committing offsets re-executes the batch. The remedy is to make the write idempotent, typically by keying it, so a repeated delivery is indistinguishable from a single one [4]. Every sink here does that

(Section 3.2.3).

Finally, Kafka guarantees ordering *within* a partition and not across them [10]. That single sentence is why partitioning is a correctness decision in this project rather than a throughput one (Section 3.1.1).

## 2.2 The Decision

We implement a **Lambda architecture**: an immutable master dataset written by a dedicated archiver, a *speed layer* computing approximate live views, and a *batch layer* recomputing exact views from the complete archive (Figure 2.1). The decision follows from Table 1.1 rather than from a preference for a named architecture: two consumers need different guarantees from the same facts, and a single processing model must either under-serve one or lie to the other.

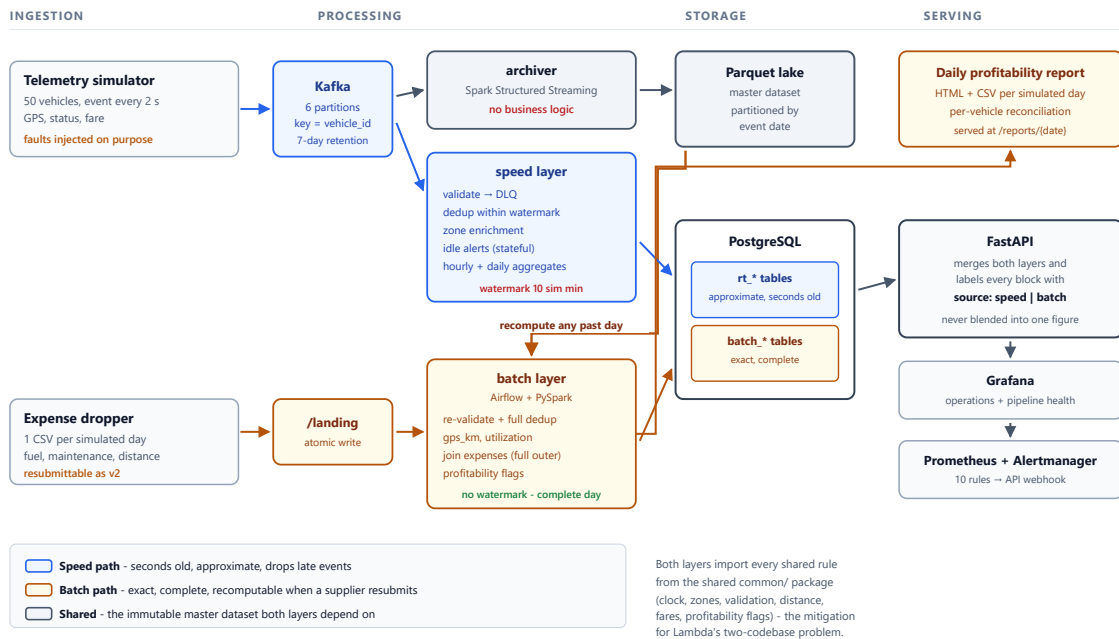


Figure 2.1: System architecture. The blue path is the speed layer (seconds old, approximate); the amber path is the batch layer (exact, complete, recomputable). Both depend on the same immutable master dataset, and both import every shared business rule from `common/`.

## 2.3 Why Kappa Was Rejected

Kappa proposes a single streaming layer, with recomputation achieved by replaying the event log [5]. It is the more fashionable choice, it is genuinely simpler to operate, and its critique of duplicated logic is correct. It has earned a specific rejection rather than a wave of the hand.

The argument that decides it is recomputation cost. Our requirement is not to reprocess data. It is to *restate a past day's financial figures* when a partner sends corrected costs. Under Kappa that implies three things:

1. **Retention must cover any day we might restate.** A partner correcting last month's file forces months of Kafka retention purely to enable periodic recomputation. Our actual retention is **7 days**, covering only speed-layer recovery

and archiver catch-up.

2. **Replay is not selective.** Recomputing one day means re-reading the log from at least that day’s start. A Parquet lake partitioned by `sim_date` lets the batch job read exactly the one partition it needs; the measured `compute_vehicle_day` task takes **9.4 seconds** for a full simulated day.
3. **The correction is not in the stream at all.** The expense file arrives on a *different channel*, as a CSV drop, once per day, long after the telemetry it relates to. There is no event log to replay that would incorporate it. Kappa’s recomputation story does not address the thing that actually forces a restatement.

The third point is the structural one. A Kappa purist could publish the expense file onto a topic and treat it as a second stream. We considered that. It moves the problem rather than solving it, because restating January from a correction that arrives in March still needs the January *telemetry* to be replayable, which is point 1 wearing a different hat. A batch-only architecture was rejected for the obvious reason: it cannot answer “which vehicles are idle right now”.

## 2.4 What Lambda Costs, and Our Mitigation

The standard criticism of Lambda is two codebases that drift [5]. We think the criticism is right, so we answered it with structure instead of a promise to be careful. A `common/` package holds every rule both layers need: the simulated clock, zone mapping, event validation, haversine distance, fare maths, and all seven profitability flags. The simulators import it, both processing layers import it, and so does the serving API. Writing a business rule twice counts as a defect here.

Two places do duplicate, and both are duplicated on purpose to avoid the most expensive thing this pipeline could do, which is running a Python UDF per row over every event. Event validation exists as `validate_event()` and as `spark_validation_expr()`; haversine distance exists as `geo.haversine_km()` and as inline Spark SQL. Each pair is held together by a test that pushes identical inputs through both and asserts identical outputs (`test_spark_validation_matches_the_python_rules` and `test_gps_km_matches_the_python_haversine`). Those tests are the only reason the duplication is defensible. Without them we would have exactly the failure Kreps describes.

## 2.5 Technology Stack

Table 2.1 gives each choice with the alternative it beat, tied to this use case’s constraints rather than to general popularity. Every image tag and Python package is pinned; there is no `latest` anywhere in the project.

## 2.6 The Simulated Clock and the Data Contract

Everything happens on a compressed timeline, where **one real second is one simulated minute** (`COMPRESSION=60`). A simulated day takes 24 real minutes, which is the only reason a genuinely *daily* reconciliation can be shown inside a lab session. The clock comes from a persisted start instant rather than a running counter, so every service computes the same `sim_time` without coordinating, and a restarted container picks up the same timeline instead of rewinding. Simulated day 1 starts at 06:00, so it is a partial day.

Table 2.1: Technology selection, with the rejected alternative for each layer.

| Layer          | Choice                            | Why this, not the alternative                                                                                                                                                                                                                                                                                                    |
|----------------|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Ingestion      | Kafka 3.8 (KRaft)                 | Partitioned ordering is a <i>correctness</i> requirement here, not a throughput one (Section 3.1.1). KRaft removes the ZooKeeper dependency, halving the moving parts on a 16 GB laptop.                                                                                                                                         |
| Speed layer    | Spark Structured Streaming 3.5.3  | Event-time windowing, watermarks and <code>applyInPandasWithState</code> in one engine [11]. Taken on its own, Flink is probably the better stream processor [12]. But using <i>one</i> engine for both layers is what makes <i>common/</i> shareable, and that shared package is our whole answer to Lambda’s central weakness. |
| Master dataset | Parquet on a volume               | Columnar, partitioned by event date, read selectively [13]. MinIO was the intended object store but its community images are no longer published; every path is built from <code>LAKE_ROOT</code> , so an <code>s3a://</code> bucket is a one-variable substitution.                                                             |
| Batch          | Airflow 2.10 + PySpark local mode | Airflow gives retries, dependency ordering and per-task observability [14]. Local mode because one simulated day is roughly 12k events, which a single JVM processes in 9 seconds; client-mode submission would add routing problems for no measurable gain.                                                                     |
| Serving store  | PostgreSQL 16                     | Both layers’ views live side by side in one queryable store [15], and upserts on natural keys give idempotent sinks (Section 3.2.3). Cassandra would suit the write-heavy <code>rt_*</code> tables but not the analytical scans over <code>batch_vehicle_daily</code> , and running both was unjustified at this volume.         |
| API            | FastAPI                           | Pydantic response models                                                                                                                                                                                                                                                                                                         |

We added three fields to the telemetry schema. `event_id` (UUID) gives deduplication something stable to key on, which matters because a duplicate is by construction *valid* and validation can never catch it. `schema_version` lets a future v2 producer coexist with v1 consumers. `event_type` (`ping`/`trip_start`/`trip_end`) is the structural one. Without an explicit lifecycle marker, counting trips means detecting a status *transition* and then aggregating over it, which is a chained stateful operator, and Structured Streaming will not allow that [11]. With the marker, “trips completed” is a plain count(`WHERE event_type='trip_end'`), identical in both layers.

One reading of an existing field matters more than any of the additions. `fare` is **cumulative within a trip and final only on** `trip_end`, and zero everywhere else. Revenue is therefore `SUM(fare) WHERE event_type='trip_end'` in both layers. Sum `fare` across all rows instead and you multiply revenue by roughly the number of pings per trip. That is the easiest way to get this pipeline’s headline number wrong, so the rule carries a test in the simulator, the speed layer and the batch layer.

## Chapter 3

# Implementation

### 3.1 Ingestion

#### 3.1.1 Partitioning Is a Correctness Decision

The topic `fleet.telemetry` has 6 partitions, **keyed by** `vehicle_id`. Throughput has nothing to do with it. Kafka guarantees ordering only *within* a partition [10], and the speed layer’s per-vehicle state machine, which tracks when each vehicle last went idle, assumes it sees one vehicle’s events in the order they happened. Keying by `vehicle_id` is what gives it that. Key round-robin instead and every idle measurement quietly goes wrong, without anything anywhere raising an error. The dead-letter topic `fleet.telemetry.dlq` has one partition, since it needs no ordering and one partition keeps every rejected event in a single readable stream.

#### 3.1.2 Fault Injection

The simulator corrupts its own output, so we can show the error handling working rather than claim it does. Table 3.1 gives configured against measured rates over roughly 2,700 events. Validation rejects rather than repairs: coerce a malformed field and you get a confidently wrong money figure downstream, which is worse than a rejection somebody can see. The DLQ carries the reason code *and the original payload*, so once a producer is fixed its rejected events can be replayed. The observed distribu-

tion was `unparseable_json` 58, `coords_out_of_bounds` 23, `bad_status` 17, `negative_fare` 12 and `negative_speed` 10.

Table 3.1: Injected fault rates, configured against measured, and what each fault demonstrates.

| Fault                           | Config. | Measured | What it proves                                                                              |
|---------------------------------|---------|----------|---------------------------------------------------------------------------------------------|
| Malformed JSON                  | 0.5%    | 0.40%    | The archiver preserves unparseable bytes; the speed layer dead-letters rather than crashing |
| Schema-invalid                  | 0.5%    | 0.77%    | <code>common/validation.py</code> rejects identically in both layers                        |
| Duplicate <code>event_id</code> | 1.0%    | 1.03%    | Deduplication works; both copies are <i>valid</i> , so nothing else can stop them           |
| Late by 1–20 sim min            | 2.0%    | 2.09%    | The consistency argument; see Section 4.1                                                   |

A second simulator drops one expense CSV per simulated day. Files are versioned, so a partner correcting an earlier submission drops a v2 file for a date already reconciled. That is the event the whole architecture exists to absorb (Section 4.2). The delivery SLA governs first delivery only. An early version checked it against every version, which meant a corrected v2 marked every resubmission late.

## 3.2 The Speed Layer

### 3.2.1 The Archiver Does as Little as Possible

The archiver is a *separate Spark application* with its own checkpoint and **no business logic at all**. Every past day is recomputed from the master dataset, so if the master dataset is corrupt then all history is wrong and nothing can recover it. Keeping the archiver separate means a bug in the speed layer’s validation, windowing or state handling *cannot reach it*. The worst such a bug can do is produce wrong dashboards, and the next batch run overwrites those. Unparseable records are archived under `sim_date=unknown` with their raw bytes intact. An archive that quietly dropped them would make “how much bad data did we receive?” unanswerable forever.

### 3.2.2 Stateful Idle Alerts

No window or aggregation can answer “has this vehicle been idle for 45 simulated minutes?”. Answering it needs memory of *when* the vehicle last stopped being idle, carried across micro-batches, plus the ability to fire when no new event arrives at all, since a silent vehicle is exactly the case we care about. `applyInPandasWithState` is the only Structured Streaming construct that offers both [11]. Four details decide whether it works, and each is a defect if you get it wrong:

1. **The stopwatch starts on the *transition* into idle**, not on every idle ping. Resetting per ping means the timer never advances and no alert can fire.
2. **Events are sorted by event time before folding**, because a micro-batch may interleave partitions.
3. **A stale out-of-order event does not rewind the state**. The event is not lost, since the archive holds it and the batch layer counts it, but rewinding would corrupt the idle measurement.

4. `EventTimeTimeout`, **not** `ProcessingTimeTimeout`. A processing-time timeout fires after  $N$  *real* seconds, so changing the clock compression would silently change the business rule.

This design has a consequence we would rather state than hide. An event-time timeout fires only when the watermark advances, and the watermark advances only when other events arrive [6]. If the whole fleet went silent at once, no timeout would fire at all. A different control covers that case: the `TelemetryNotProduced` alert, which watches the producer directly.

### 3.2.3 Idempotent Sinks

`foreachBatch` gives **at-least-once** delivery. If the driver dies after writing a micro-batch but before committing offsets, Spark re-runs the batch and the sink sees the same rows twice [8]. With plain inserts, every dashboard figure would ratchet upward on each restart and never come back down. Every sink therefore upserts on a natural key: `rt_vehicle_state` on `(vehicle_id)`, `rt_zone_hourly` on `(zone_id, window_start)`, `rt_vehicle_daily` on `(vehicle_id, sim_date)`, and `idle_alerts` on `(vehicle_id, opened_at)`. A repeated write then sets the same values it set the first time [4]. It also lets the windowed aggregations use `update` output mode, which they need: under `append`, a window is withheld until the watermark passes its end, so the *current* simulated hour, the one thing a live dashboard exists to show, would never appear.

## 3.3 The Batch Layer and Orchestration

### 3.3.1 The Batch Layer Trusts Nothing

`compute_vehicle_day` re-reads the raw archive and re-applies the same validation and deduplication from scratch. It never reads the speed layer's output, and it applies **no watermark**. It is reading a complete, closed day, so lateness means nothing to it, and it deduplicates across the whole day instead of within a watermark window. That independence is the only reason the drift measurement in Section 4.1 means anything: the two figures really are arrived at twice. Its outputs per vehicle-day are `trips`, `revenue`, `gps_km` (haversine between consecutive pings, discarding jumps above 150 km/h), `online_min`, `on_trip_min`, `idle_min` and `utilization`.

The reconciliation joins telemetry to expenses with a full outer join, and the choice is not accidental. An inner join would quietly drop the two findings the business most needs to see: telemetry with no expense row (`missing_costs`, meaning we are blind to a vehicle's costs) and an expense row with no telemetry (`missing_telemetry`, meaning we are being billed for a vehicle that never reported).

### 3.3.2 Schedule and Recomputation

Airflow's scheduler lives in real time and knows nothing about the compressed clock. So the DAG runs every 2 real minutes and its *first* task works out which simulated day is pending. The schedule is a poll; the date selection comes from the data. We did try expressing the schedule in simulated time, and got an interval 60 times longer than we meant. The DAG has nine tasks: `find_pending_date`, `check_expense_file`, `validate_expenses`, `compute_vehicle_day`, `reconcile_profitability`, `load_batch_views`, `compute_drift`, `generate_report` and `record_run`.

`load_batch_views` deletes every row for the target `sim_date` and inserts the re-computed set inside a single transaction, so rerunning a date produces the same row

count with updated values and a partially applied recompute is impossible. This is the property the architecture exists to provide, demonstrated in Section 4.2.

## 3.4 Storage and Serving

Both layers' outputs live in one PostgreSQL instance, every `rt_*` table keyed on its natural key so the streaming upserts work (Appendix B). The API exposes eleven documented endpoints (Appendix D), the consolidated daily report is served at `GET /reports/{sim_date}`, and two Grafana dashboards read the same tables.

### 3.4.1 The Lambda Merge Is Explicit, Never Blended

`GET /vehicles/{id}` returns three blocks, each labelled with its own `source` and `as_of`: `live` and `today` carry `"source": "speed"`, while `history` carries `"source": "batch"`. We do not merge them into a single figure. The two layers answer different questions under different guarantees, and a consumer has to be able to see at a glance which numbers are safe to quote to finance. Blending them is how a dashboard estimate ends up in a financial statement. “Now” is always *simulated* now, taken from `max(last_event_time)` rather than the database clock, so if ingestion stalls then “now” stops advancing and the active-vehicle count drops to zero instead of quietly lying.

## 3.5 Observability

Four questions drive every metric. Nothing is collected merely because it was easy to collect.

*Is data arriving?* `fleet_events-produced_total` and `fleet_faults-injected_total` answer that, and between them they separate “the producer is silent” from “the producer is gone”, which is why the alert tests both `increase()==0` and `absent()`. *Is it being processed?* `fleet_stream-last-progress-timestamp` and its siblings, all recorded per query rather than per process, because a container can look perfectly healthy while one of its five queries sits frozen. *Is it correct?* `fleet_dlq-events-total`, `fleet_late-rows-dropped-total` and `fleet_speed-batch-drift-ratio`, which measure imprecision we know about and have accepted, on the principle that a pipeline unable to quantify its own error has no business handling money. *Is the business healthy?* `fleet_open-idle-alerts` and the profitability flags, kept well away from pipeline health, because `IdleVehiclesHigh` fires when everything works and the fleet simply is not earning.

Every log line is one JSON object carrying `ts`, `level`, `service`, `stage` (`ingestion/processing/storage/serving/orchestration`), `event`, `msg` and `sim_time`. That last field matters most here, because on a compressed clock a wall-clock timestamp tells you nothing about *which simulated hour* a line belongs to. Per-event logging is sampled 1-in-*N*: at 50 vehicles emitting every 2 seconds, an INFO line per event buries everything else, so the useful granularity is one line per micro-batch carrying counts.

We do not implement distributed tracing at all. What we implement is **correlation-ID propagation**. `event_id` travels producer → Kafka → archiver → DLQ → batch dedup. `vehicle_id` travels through every stage and doubles as the partition key. `run_id` tags every Airflow task and batch table, and `sim_date` identifies the lake partition and the report filename. So a rejected event can be traced from its DLQ reason back to its original payload, and a reconciled figure back to the run and

expense-file version behind it. What we give up is per-request span timing across service boundaries. In a pipeline whose hops are Kafka offsets and Parquet partitions rather than RPC calls, spans would add infrastructure without answering a question we actually have. Section 4.4.3 says when that stops being true.

### 3.5.1 Alerting Decisions Worth Defending

**Kafka lag comes from Spark’s own query progress, not from a lag exporter.** Structured Streaming never commits consumer-group offsets. It tracks them in its checkpoint instead, which means `kafka-consumer-groups.sh` and every off-the-shelf exporter report *nothing* for these consumers. The number exists in one place only: the progress event’s source metrics.

**Batch metrics are exported by the API from PostgreSQL, not pushed.** Airflow tasks are short-lived and Prometheus pulls, so by the time a scrape happens the task has exited. A Pushgateway would work, but it is one more service holding job state indefinitely with no notion of staleness, and the Prometheus documentation discourages it for exactly this pattern [17]. The batch layer already persists its outcomes because the API and the report need them, so re-exporting costs us nothing and means the metric and the report cannot disagree.

Ten alert rules are defined (Appendix C), each with a `for:` duration, a severity and a runbook entry. Section 4.3 reports which have been verified firing and delivered.

## Chapter 4

# Results and Discussion

Every figure below was read off the running stack, not estimated. We started the system from a fresh clone with `make up` and left it alone; the Airflow DAG reconciled four simulated days without anyone touching it. The metrics come from the Prometheus endpoints and the PostgreSQL bookkeeping tables, which are the same sources the dashboards and the daily report read.

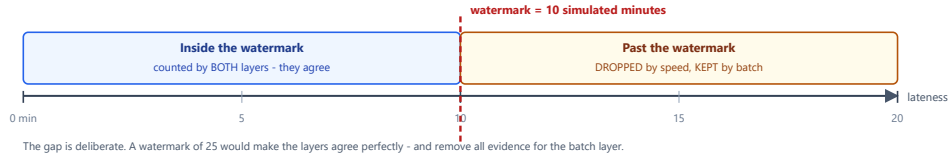
### 4.1 The Consistency Argument

This section is where the architecture earns its keep. The speed layer applies a 10-simulated-minute watermark while the simulator injects lateness of up to 20 simulated minutes, and we left that gap open on purpose. Set the watermark to 25 and it catches everything, the two layers agree perfectly, and the project has no evidence left for why the batch layer exists at all. A regression test fails if anyone raises it.

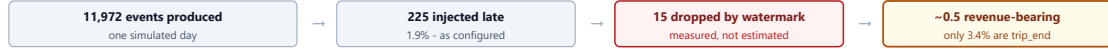
On 2024-01-02 the drift was 0.34%, with `speed_trips` = 316 against `batch_`

**An event's lateness decides which layer counts it**

Lateness is injected uniformly between 1 and 20 simulated minutes. The speed layer's watermark is 10.

**Why the revenue drift is usually zero anyway**

Measured over one simulated day. Each step narrows the number of events that can actually move the money figure.



15 dropped rows  $\times$  3.4% `trip_end` share = **half of one** revenue-bearing event per day. Zero drift is therefore the single most likely daily outcome.

**Measured across four simulated days**

| Simulated day     | Speed revenue    | Batch revenue    | Drift           | Speed trips | Batch trips | Interpretation                                  |
|-------------------|------------------|------------------|-----------------|-------------|-------------|-------------------------------------------------|
| 2024-01-01        | 49,434.23        | 49,434.23        | 0.000000        | 378         | 378         | no trip dropped                                 |
| <b>2024-01-02</b> | <b>41,088.10</b> | <b>41,227.61</b> | <b>0.003384</b> | <b>316</b>  | <b>317</b>  | <b>one late <code>trip_end</code> recovered</b> |
| 2024-01-03        | -                | -                | 0.000000        | 428         | 428         | no trip dropped                                 |
| 2024-01-04        | -                | -                | 0.000000        | 402         | 402         | no trip dropped                                 |

Figure 4.1: Why the two layers disagree, and by how much. An event's lateness decides which layer counts it; the funnel shows why that translates into a near-zero revenue drift on a typical day.

Table 4.1: Reconciliation outcomes over four consecutive simulated days. Revenue drift is the relative difference between the speed layer's and the batch layer's daily revenue.

| Simulated day        | Fleet margin | Unprof. | Becoming unprof. | Dist. mismatch | Revenue drift   |
|----------------------|--------------|---------|------------------|----------------|-----------------|
| 2024-01-01 (partial) | 42.8%        | 6       | 0                | 2              | 0.000000        |
| 2024-01-02           | 38.3%        | 4       | 0                | <b>21</b>      | <b>0.003384</b> |
| 2024-01-03           | 45.8%        | 3       | 6                | 6              | 0.000000        |
| 2024-01-04           | 47.7%        | 4       | 7                | 4              | 0.000000        |

`trips` = 317. One late `trip_end` event, dropped by the watermark and recovered by the batch layer. That single row is the clearest evidence in the project, because the mechanism is countable and we can point at the event behind it.

We had expected a visibly non-zero drift every day and did not get one. Working out why turned into a better argument than a dramatic number would have been. Over one simulated day the producer emitted 11,972 events and injected 225 late ones (1.9%), the watermark dropped **15 rows**, and 3.4% of all events are `trip_end`. So the number of dropped revenue-bearing events we should expect per day is  $15 \times 0.034 \approx 0.5$ , which is to say about half of one. On any given day zero drift is the most likely single outcome, with non-zero turning up occasionally, and that is what Table 4.1 shows.

The two metrics do different jobs and the report leans on both. `fleet_late_rows_dropped_total` shows the *mechanism*, and it is reliably non-zero. `fleet_speed_batch_drift_ratio` shows the *financial weight* of that mechanism, and it sits near zero most days. Either one on its own misleads: the first makes the problem look bigger than it is, the second makes it look like nothing is happening.

Our case for computing money in the batch layer was never that the speed layer is wildly wrong. It is that **the speed layer's error is unbounded in principle and unknowable in advance**. A watermark is a bet on how late data arrives [6], and most days that bet is won comfortably. A partner resubmitting a corrected file

for last month is a different thing entirely, and no watermark value answers it. Only recomputation from an immutable archive does. We could have inflated the injected fault rates and reported a bigger number. We did not. Reporting 0.000000 next to the arithmetic above is more defensible than reporting 8% after quietly editing the simulator.

## 4.2 Recomputation Demonstrated

Two experiments test the property the whole architecture exists to provide. First, we dropped a `v2` expense file for a day already reconciled. The day went pending again and the recompute produced **the same 50 rows with different values**, net profit moving from 19,770.78 to 21,177.99, with the quarantined-row count falling from 1 to 0. Second, we triggered the DAG manually with `{"sim_date": "2024-01-03", "force": true}`. It recomputed that date to **bit-identical figures** (net 24,902.95) under a fresh `computed_at` timestamp, so recomputation here is deterministic, not just repeatable.

That is Section 2.3’s argument in concrete form. The correction arrived on the file channel, and the restatement came out of re-reading a single Parquet partition, with no log replay involved at any stage.

## 4.3 Verification

Ten alert rules are defined, and **five have been watched firing *and* arriving** end to end through Alertmanager into the `alert_notifications` table (Appendix C). A sixth, `BatchRunFailed`, fired and was stopped on purpose. A missing partner file makes the reconciliation record a failed run, so the rule fired alongside `ExpenseFileLate` for a single root cause, and an Alertmanager inhibit rule suppresses it (the API confirms this with `inhibitedBy = 1`). Sending two alerts for one cause, where the second one misleads, is how a team learns to ignore its alert channel [19]. Nothing was broken in that scenario; we were waiting on a supplier. Batch task durations came out at `compute_vehicle_day` 9.44s, `load_batch_views` 0.99s, `reconcile_profitability` 0.45s, and everything else under 0.04s.

Everything claimed here was run against the live stack: 202 unit, PySpark and API tests pass; the end-to-end smoke test passes 28 of 28 checks; a fresh clone reaches all 13 services healthy and serving live data on a single command; four simulated days reconciled cleanly at a 38–48% fleet margin; 5 of 10 alert rules are verified firing and delivered, with a sixth intentionally inhibited; and 23 of 23 Grafana panel queries return live data.

## 4.4 Discussion

### 4.4.1 What Running It Taught Us

We found six defects by running the system, not by reading the code. None of them was visible in review. A Spark worker restart killed the archiver’s executor while the query stayed “active”, so `awaitTermination()` never returned, **the healthcheck stayed green**, and the archiver stopped writing the master dataset for 26 minutes without saying a word. The batch layer then reconciled a day from that incomplete archive. Separately, the odometer ledger checkpointed every 30 *real* seconds, which

at `COMPRESSION=60` is 30 *simulated* minutes, so a restart of our own making flagged 21 of 50 vehicles for a distance mismatch no partner had caused. And the first full reconciliation called 35 of 50 vehicles unprofitable. The pipeline was right the whole time; the economics simply had not been calibrated. Appendix F lists all six with the lesson each one carries.

#### 4.4.2 Limitations

Set out plainly, since nobody believes a report that claims no weaknesses.

- **Single-run measurements.** Four days is one run of one configuration, and the drift figures are small integer counts dominated by the arrival of late `trip_end` events. The arithmetic in Section 4.1 is there so a reader can judge the expected variance instead of over-reading four samples. Day 1 is also partial (06:00 start), so its 42.8% margin is not comparable with the rest.
- **The revenue drift is usually zero.** You can see the mechanism in `fleet_late_rows_dropped_total`. You usually cannot see its financial impact. Anyone hoping for a dramatic number will be disappointed.
- **Some findings need more days than a demo has.** `becoming_unprofitable` wants three reconciled days before it will fire. That is the behaviour we want, since a trend drawn from two points is not a trend, but it does constrain what a short demo can show. “Lemon” vehicles have the same problem: they are flagged more often, not always, at 25% of vehicle-days against 12.8% for the rest of the fleet.
- **Batch Spark runs in local mode**, so distributed *batch* is designed for but never demonstrated. And with **one Kafka broker at RF 1**, losing the broker means losing anything not yet archived.
- **Downtime creates real data gaps.** Simulated time comes from wall-clock time, so while the stack is stopped, simulated hours keep passing with no events emitted. A fifth simulated day was measurably degraded by our own outage testing. The pipeline reported the reduced revenue faithfully instead of inventing something, which is the behaviour we want, but it does mean operational downtime looks exactly like low demand in the data.

#### 4.4.3 At Production Scale

Every simplification above exists because this is a two-week project on one laptop. Appendix G pairs each with what a real fleet would need. Three changes matter more than the rest.

Ingestion would move to three brokers at RF 3 behind a schema registry, so an incompatible producer is rejected at publish time instead of being discovered later in the dead-letter queue. The *keying* would stay as it is, because that is a correctness constraint and not a tuning knob. Processing would move to Kubernetes, and the change worth making first is pulling the archiver off the same worker as the speed layer. They already have separate checkpoints, but they compete for the same cores, and that competition is how we lost 26 minutes of archiving. Observability would gain log aggregation before anything else, which is a deployment step rather than a code change, since the logs are already structured JSON carrying a `stage` field. Real tracing would come next, and SLOs with error budgets after that. A threshold like `IdleVehiclesHigh > 8` is tuned to a 50-vehicle fleet and will not survive that fleet doubling [19].

## Chapter 5

# Conclusions and Recommendations

The brief’s business question is really two questions, and their latency, accuracy and recomputability requirements do not fit together. What justifies Lambda here is not convention but one property Kappa cannot supply cheaply: **restating a past day’s financial figures when a correction arrives on a different channel, days later**. We showed that property working. A corrected file triggers an idempotent recompute producing the same 50 rows with updated values, and a forced replay returns bit-identical figures.

We also put a number on the speed layer’s approximation instead of asserting one, then explained arithmetically why it is as small as it is. A measured drift of 0.000000 on three days of four, reported next to the arithmetic that predicts it, beats a bigger number obtained by quietly raising the fault rate. Lambda’s usual weakness, two codebases drifting apart, we answered with the shared `common/` package and tests that hold its two duplications in step. For a reader who arrives persuaded by Kreps, that is the part we would defend hardest.

## Recommendations

In priority order, and independent of scale:

1. **Decouple the archiver from the speed layer** onto separate workers. They already have separate checkpoints; sharing cores is what turned a worker restart into 26 minutes of lost archiving.
2. **Add a schema registry** in front of the telemetry topic, so a bad producer is rejected at publish time instead of discovered in the dead-letter queue, and give the DLQ an automated replay path.
3. **Adopt a lake table format** (Iceberg or Delta Lake), to give lake writes the atomicity delete-then-insert already gives the PostgreSQL views.
4. **Annotate outage windows**, so the `becoming_unprofitable` rule can exclude degraded days instead of reading them as a commercial decline.
5. **Express alert thresholds as ratios**, so that `IdleVehiclesHigh` survives a change in fleet size.

The most valuable thing we got out of this was not the architecture. It was what running it turned up: a green healthcheck over a dead archiver, a correct pipeline producing meaningless economics, an interval 60× longer than we intended. None of it visible in code review.

# References

- [1] Department of Electrical and Information Engineering, “EC8203 applied big data engineering: Data engineering mini-project assessment.” Faculty of Engineering, University of Ruhuna, 2026.
- [2] N. Marz and J. Warren, *Big Data: Principles and Best Practices of Scalable Real-Time Data Systems*. Shelter Island, NY: Manning Publications, 2015.
- [3] P. Helland, “Immutability changes everything,” in *Proceedings of the 7th Biennial Conference on Innovative Data Systems Research (CIDR)*, 2015.
- [4] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. Sebastopol, CA: O’Reilly Media, 2017.
- [5] J. Kreps, “Questioning the lambda architecture.” O’Reilly Radar, 2014. <https://www.oreilly.com/radar/questioning-the-lambda-architecture/>. Accessed 2026-09-27.
- [6] T. Akidau, R. Bradshaw, C. Chambers, S. Chernyak, R. J. Fernández-Moctezuma, R. Lax, S. McVeety, D. Mills, F. Perry, E. Schmidt, and S. Whittle, “The dataflow model: A practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing,” *Proceedings of the VLDB Endowment*, vol. 8, no. 12, 2015.
- [7] T. Akidau, A. Balikov, K. Bekiroğlu, S. Chernyak, J. Haberman, R. Lax, S. McVeety, D. Mills, P. Nordstrom, and S. Whittle, “MillWheel: Fault-tolerant stream processing at internet scale,” *Proceedings of the VLDB Endowment*, vol. 6, no. 11, 2013.
- [8] M. Armbrust, T. Das, J. Torres, B. Yavuz, S. Zhu, R. Xin, A. Ghodsi, I. Stoica, and M. Zaharia, “Structured streaming: A declarative API for real-time applications in Apache Spark,” in *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*, 2018.
- [9] M. Zaharia, T. Das, H. Li, T. Hunter, S. Shenker, and I. Stoica, “Discretized streams: Fault-tolerant streaming computation at scale,” in *Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP)*, 2013.
- [10] J. Kreps, N. Narkhede, and J. Rao, “Kafka: A distributed messaging system for log processing,” in *Proceedings of the 6th International Workshop on Networking Meets Databases (NetDB)*, 2011.

- [11] Apache Software Foundation, “Structured streaming programming guide, Apache Spark 3.5.3.” <https://spark.apache.org/docs/3.5.3/structured-streaming-programming-guide.html>, 2024. Accessed 2026-09-27.
- [12] P. Carbone, A. Katsifodimos, S. Ewen, V. Markl, S. Haridi, and K. Tzoumas, “Apache Flink: Stream and batch processing in a single engine,” *IEEE Data Engineering Bulletin*, vol. 38, no. 4, 2015.
- [13] Apache Software Foundation, “Apache Parquet file format specification.” <https://parquet.apache.org/docs/>, 2024. Accessed 2026-09-27.
- [14] Apache Software Foundation, “Apache Airflow documentation, version 2.10.” <https://airflow.apache.org/docs/>, 2024. Accessed 2026-09-27.
- [15] The PostgreSQL Global Development Group, “PostgreSQL 16 documentation.” <https://www.postgresql.org/docs/16/>, 2024. Accessed 2026-09-27.
- [16] S. Ramírez, “FastAPI documentation.” <https://fastapi.tiangolo.com/>, 2024. Accessed 2026-09-27.
- [17] Prometheus Authors, “Prometheus documentation: Querying, alerting rules and the pushgateway.” <https://prometheus.io/docs/>, 2024. Accessed 2026-09-27. The Pushgateway guidance cited in Section 4.5 is from *When To Use The Pushgateway*.
- [18] Grafana Labs, “Grafana documentation.” <https://grafana.com/docs/grafana/latest/>, 2024. Accessed 2026-09-27.
- [19] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, eds., *Site Reliability Engineering: How Google Runs Production Systems*. Sebastopol, CA: O’Reilly Media, 2016.

# Appendix A

## Repository Layout

Table A.1: Top-level layout of the submitted repository.

| Directory      | Contents                                                                                                                                                                                                                                                                                      |
|----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| common/        | Shared by <i>both</i> layers: <code>simclock</code> , <code>geo</code> (zones and haversine), <code>schemas</code> , <code>validation</code> , <code>earnings</code> , <code>profitability</code> , JSON logging, <code>metrics</code> , <code>db upsert helpers</code> , <code>config</code> |
| simulators/    | <code>telemetry/</code> (Kafka producer and fleet state machine), <code>expenses/</code> (daily CSV dropper), <code>control.py</code>                                                                                                                                                         |
| streaming/     | <code>archiver.py</code> , <code>speed.py</code> , <code>state.py</code> , <code>sinks.py</code> , <code>listener.py</code> , <code>watchdog.py</code>                                                                                                                                        |
| batch/         | <code>runs.py</code> , <code>expenses.py</code> , <code>vehicle_day.py</code> , <code>reconcile.py</code> , <code>report.py</code>                                                                                                                                                            |
| airflow/dags/  | <code>fleet_daily_reconciliation.py</code> (9 tasks)                                                                                                                                                                                                                                          |
| api/           | FastAPI: <code>app.py</code> , <code>queries.py</code> , <code>models.py</code> , <code>batch_metrics.py</code>                                                                                                                                                                               |
| db/init/       | PostgreSQL schema, indexes and views                                                                                                                                                                                                                                                          |
| observability/ | <code>prometheus/</code> (config and 10 alert rules), <code>alertmanager/</code> , <code>grafana/</code> (provisioning and 2 dashboards)                                                                                                                                                      |
| scripts/       | <code>simclock_init</code> , <code>create_topics</code> , <code>wait_for_stack</code> , <code>demo</code> , <code>smoke_test</code> , <code>report build helpers</code>                                                                                                                       |
| tests/         | 202 tests: unit, PySpark and API                                                                                                                                                                                                                                                              |
| docs/          | <code>SPEC.md</code> , <code>this report</code> , <code>decisions.md</code> , <code>contributions.md</code> , <code>runbook.md</code> , <code>walkthrough.md</code> , <code>report-assets/</code>                                                                                             |

## Appendix B

### Database Schema

Every `rt_*` table carries a primary key on its *natural* key, which is what makes the streaming upserts of Section 3.2.3 idempotent.

Table B.1: Serving-store tables, by owning layer.

| Table                              | Key                     | Purpose                                                             |
|------------------------------------|-------------------------|---------------------------------------------------------------------|
| <i>Speed layer</i>                 |                         |                                                                     |
| rt_vehicle_state                   | (vehicle_id)            | Current status, zone and last event time per vehicle                |
| rt_zone_hourly                     | (zone_id, window_start) | Windowed trips and earnings by zone and simulated hour              |
| rt_vehicle_daily                   | (vehicle_id, sim_date)  | Running daily totals from the speed layer                           |
| idle_alerts                        | (vehicle_id, opened_at) | Threshold alerts raised by the stateful idle machine                |
| <i>Batch layer and bookkeeping</i> |                         |                                                                     |
| batch_vehicle_daily                | (vehicle_id, sim_date)  | Reconciled per-vehicle-day profitability and its seven flags        |
| batch_runs                         | (run_id)                | One row per reconciliation attempt, with status and expense version |
| dq_issues                          | (id)                    | Quarantined rows with reason, by date                               |
| speed_batch_drift                  | (sim_date)              | Per-date speed-versus-batch revenue comparison                      |
| alert_notifications                | (id)                    | Alerts delivered by Alertmanager to the API webhook                 |

Table C.1: The ten Prometheus alert rules. *Verified* means observed firing **and** delivered into `alert_notifications`.

| Alert                | Stage         | Sev.     | Condition                                     | Verified |
|----------------------|---------------|----------|-----------------------------------------------|----------|
| TelemetryNotProduced | ingestion     | critical | No events produced for over a minute          | yes      |
| DLQRateHigh          | ingestion     | warning  | More than 2% of events dead-lettered          | yes      |
| StreamStalled        | processing    | critical | A named query has stopped progressing         | –        |
| StreamLagHigh        | processing    | warning  | Query more than 5,000 off-sets behind         | –        |
| ExpenseFileLate      | orchestration | warning  | An expense file missed its first-delivery SLA | yes      |
| BatchRunFailed       | orchestration | critical | Last reconciliation run inhibited failed      | –        |
| BatchNotRunRecently  | orchestration | warning  | No successful reconciliation in 45 minutes    | yes      |
| SpeedBatchDriftHigh  | processing    | warning  | Speed and batch revenue differ by over 5%     | –        |
| ApiDown              | serving       | critical | The serving API is not reachable              | yes      |
| IdleVehiclesHigh     | serving       | warning  | More than 8 vehicles idle past threshold      | –        |

## Appendix C

### Alert Rule Inventory

Each rule carries a `for:` duration of 1–2 real minutes. The durations are short on purpose: at `COMPRESSION=60`, one real minute is one simulated hour, so a conventional five-minute `for:` would mean waiting five simulated hours before an alert fired. Every rule has a corresponding entry in `docs/runbook.md` giving what it means, how to reproduce it and how to fix it.

## Appendix D

### API Surface

Table D.1: The eleven documented endpoints, by the layer that answers them.

| Endpoint                   | Layer | Answers                                                                                                                        |
|----------------------------|-------|--------------------------------------------------------------------------------------------------------------------------------|
| GET /health                | ops   | Liveness, plus <code>sim_time</code> and clock compression                                                                     |
| GET /fleet/live            | speed | Active vehicles, idle ratio, trips/hour, earnings                                                                              |
| GET /zones/live            | speed | Current utilisation and earnings by zone                                                                                       |
| GET /zones/{id}/hourly     | speed | One zone's windowed history by simulated hour                                                                                  |
| GET /alerts/idle           | speed | Open and closed idle alerts                                                                                                    |
| GET /vehicles/unprofitable | batch | Vehicles flagged unprofitable or becoming unprofitable                                                                         |
| GET /vehicles/{id}         | both  | The explicit Lambda merge: <code>live</code> and <code>today</code> from speed, <code>history</code> from batch, each labelled |
| GET /reports/{sim_date}    | batch | The consolidated daily reconciliation report (HTML)                                                                            |
| GET /pipeline/status       | ops   | Reconciled dates, last run, drift, expense-file state                                                                          |
| POST /alerts/webhook       | ops   | Alertmanager delivery target; writes <code>alert_notifications</code>                                                          |
| GET /metrics               | ops   | Prometheus exposition, including re-exported batch metrics                                                                     |

## Appendix E

# Statement of Individual Contributions

The project was delivered by a three-member group. The work was divided by **pipeline stage**, so that each member owned one end-to-end vertical slice — its source code, its schema objects, its tests and its sections of this report — rather than all three editing the same files. Table E.1 records the principal areas each member led. Design decisions were taken jointly and code was reviewed across the team.

## Decisions each member defends

**Ahamed N.F.F.** Partitioning by `vehicle_id` as an ordering requirement (Section 3.1.1); the  $60\times$  simulated clock and its derivation from a persisted start instant (Section 2.6); the deliberate fault rates (Section 3.1.2); and the decision that validation rejects rather than repairs.

**Safeer S.M.** The deliberately boring archiver (Section 3.2.1); the four subtleties of the stateful idle machine and the choice of `EventTimeTimeout` (Section 3.2.2); idempotent sinks and their natural keys (Section 3.2.3); and the explicit, never-blended Lambda

Table E.1: Principal areas of responsibility by team member.

| Member        | Reg. No.     | Principal areas                                                                                                                                                                                                                                                                                                                        |
|---------------|--------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Ahamed N.F.F. | EG/2021/4390 | Ingestion and the data contract: <code>simulators/</code> (telemetry producer, fleet state machine, fault injection, expense dropper), Kafka topic setup and partitioning, and the <code>simclock</code> , <code>schemas</code> , <code>validation</code> , <code>geo</code> and <code>earnings</code> modules of <code>common/</code> |
| Safeer S.M.   | EG/2021/4771 | Stream processing and serving: <code>streaming/</code> (archiver, speed layer, per-vehicle state machine, sinks, listener, watchdog), <code>api/</code> , the <code>rt_*</code> tables, and the db upsert helpers in <code>common/</code>                                                                                              |
| Nafla M.N.P.  | EG/2021/4687 | Batch, orchestration and observability: <code>batch/</code> , the <code>fleet_daily_reconciliation</code> DAG, all of <code>observability/</code> (Prometheus rules, Alertmanager, both Grafana dashboards), and the <code>profitability</code> , <code>metrics</code> and <code>logging</code> modules of <code>common/</code>        |

merge (Section 3.4.1).

**Nafla M.N.P.** The batch layer trusting nothing and applying no watermark (Section 3.3.1); delete-then-insert recomputation (Section 4.2); the poll-not-cron DAG schedule; and the alerting design, including the inhibition rule (Section 3.5.1).

## Deliberately shared: the `common/` package

`common/` is the project’s answer to Lambda’s central weakness (Section 2.4). Zone mapping, distance, fare, validation and profitability rules are defined *once* and imported by the simulator, both processing layers and the API. Each module has a primary author, listed above, but the rule the team worked to was that no business rule is defined twice, and a change to a `common/` module is reviewed by whichever members’ layers import it. That constraint is why every member can explain modules they did not personally write: their own layer depends on them.

## Worked on jointly

`docs/SPEC.md`, written together before any code and treated as fixed thereafter; `docker-compose.yml`, the `Makefile` and `common/config.py`, since every member’s service is defined in them; the smoke test, to which each member contributed the checks for their own layer; the architecture argument of Chapter 2 and the consistency analysis of Section 4.1; and every acceptance run — the fresh-clone start, the multi-day reconciliation and the alert-firing verification each require all three layers alive at once.

A fuller per-member breakdown, including the test files each member wrote, is kept in `docs/contributions.md` in the repository.

## Appendix F

### Defects Found by Operating the System

Summarised in Section [4.4.1](#). Each was invisible to code review and visible within minutes of checking the numbers against expectations.

## Appendix G

### At Production Scale

Summarised in Section [4.4.3](#). Every row is a deliberate simplification made for a two-week, single-laptop project, paired with what running a real fleet would require.

Table F.1: Defects found by operating the system, and what each generalises to.

| # | What happened                                                                                                                                                                                                                                                                                    | The general lesson                                                                                                                                                                                                                  |
|---|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | A Spark worker restart killed the archiver's executor. The query stayed "active", so <code>awaitTermination()</code> never returned, the healthcheck stayed green, and the archiver silently stopped writing for <b>26 minutes</b> ; the batch layer then reconciled from an incomplete archive. | <b>A healthcheck that proves liveness proves nothing about work.</b> <code>streaming/watchdog.py</code> now monitors <i>progress</i> and exits so Docker restarts from the checkpoint — verified by killing executors deliberately. |
| 2 | The first full reconciliation flagged <b>35 of 50</b> vehicles unprofitable, with costs at $1.9\times$ revenue. The pipeline was correct; the economics were not calibrated. After recalibration: 3–6 of 50, 38–48% margin.                                                                      | <b>A correct pipeline can produce a useless answer.</b> Only running it end to end on realistic volumes surfaced this.                                                                                                              |
| 3 | The odometer ledger checkpointed every 30 <i>real</i> seconds — 30 <i>simulated</i> minutes at <code>COMPRESSION=60</code> . A restart lost 90 km and flagged <b>21 of 50</b> vehicles for a mismatch the partner had not caused.                                                                | <b>Intervals chosen in real seconds are <math>60\times</math> longer in business terms.</b> A check that fires because of our own restart trains the reader to ignore it.                                                           |
| 4 | Batch gauges defined in <code>common/metrics.py</code> — which every service imports — meant all five services exported <code>fleet_batch_last_success_timestamp</code> at its default of 0, and <code>BatchNotRunRecently</code> fired forever on four bogus series.                            | <b>Shared code publishes shared metrics.</b> A permanently-firing false alert is worse than no alert.                                                                                                                               |
| 5 | The expense SLA was evaluated against every file version, so a corrected <code>v2</code> — by definition sent after the deadline — marked every resubmission late.                                                                                                                               | <b>An alert that fires on a normal operation is noise.</b> The SLA now governs first delivery only.                                                                                                                                 |
| 6 | The "fresh clone $\rightarrow$ one command" check exposed two Windows-only defects: PowerShell turning Docker's normal stderr progress into a terminating error, and a wrapper binding Docker's <code>-d</code> flag to itself.                                                                  | <b>Acceptance checks must actually be run.</b> No amount of reading would have found either.                                                                                                                                        |

Table G.1: Simplifications made here, and what each becomes at production scale.

| Layer         | Today                                                                                                                      | At scale                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|---------------|----------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Ingestion     | One broker, RF 1, 6 partitions, 7-day retention; <code>schema_version</code> plus defensive validation; manual DLQ replay. | Three brokers, RF 3, <code>min.insync.replicas=2</code> . A <b>schema registry</b> so incompatible producers are rejected at publish time rather than discovered in the DLQ, and an automated replay path. Partition count would follow measured throughput, but the <i>keying</i> would not change — it is a correctness constraint, not a tuning knob.                                                                                                  |
| Storage       | Parquet on a Docker volume, partitioned by <code>sim_date</code> ; archiver writes every 30 s.                             | Object storage (a one-variable change — every path is built from <code>LAKE_ROOT</code> ), plus a <b>table format</b> (Iceberg or Delta) for atomic partition overwrites, and <b>compaction</b> , without which the batch <i>read</i> outgrows the batch <i>compute</i> .                                                                                                                                                                                 |
| Processing    | Spark standalone, one worker, 3 cores, both streaming apps co-resident; batch in local mode.                               | Kubernetes with the Spark operator. The single most valuable change is <b>decoupling the archiver from the speed layer</b> — they already have separate checkpoints but compete for the same cores, which is how 26 minutes of archiving was lost (defect 1).                                                                                                                                                                                             |
| Serving       | One PostgreSQL instance for both layers, queried directly by Grafana.                                                      | The access patterns diverge: keep PostgreSQL for the <code>rt_*</code> point lookups, add read replicas so dashboards cannot slow the streaming sinks, and move <code>batch_vehicle_daily</code> to a columnar warehouse once history outgrows one node. Add caching on <code>/fleet/live</code> and authentication.                                                                                                                                      |
| Observability | Prometheus, Alertmanager, Grafana, structured JSON logs, correlation IDs.                                                  | <b>Log aggregation</b> (Loki or ELK) — the logs are already structured JSON with a <code>stage</code> field precisely so this is a deployment step, not a code change. Then <b>real tracing</b> , once multiple producers make “which producer caused this DLQ spike?” unanswerable from correlation IDs. Then <b>SLOs with error budgets</b> [19]: <code>IdleVehiclesHigh &gt; 8</code> is tuned to a 50-vehicle fleet and does not survive it doubling. |